

## CMSC411 Assignment 2

Fall 2024

**On-time Due: September 22 by 11:00 PM (EDT) / Late Due: September 23 by 11:00 PM (EDT)**

---

### General instruction

- Submit a PDF file to GradeScope – You may edit the PDF file directly, scan your work on paper, or can also use an app (e.g., Scannable) to create a PDF file from photos of your work.
- After uploading your work, **don't forget to create mapping** between questions and your answers. 5 points will be deducted if mapping is not or incorrectly given.
- Unless specified, all questions will be graded.
- Unless specified, **justify your answers always**. If justifications are missing or do not show reasonably how your answers are derived, 0 points will be assigned regardless of the correctness of the answers.
- Write **neatly** in case you write your answers on paper. **Especially, for table completion questions, if your answers are not given neatly in well-drawn tables, the answers won't be graded.** It is also recommended to print this file and put your answers on the printout.
- **This assignment must be done by yourself without any unauthorized assistance.** Write your name and numeric student ID in the space below, and put your signature after reading the pledge statement.

Name: James Zhang

---

SID (Numeric): 118843940

---

*I pledge my honor that all parts of this assignment were done by me individually and without any unauthorized assistance or collaboration with others.*

Signature: 

---

### Q1. Assemble Instruction

Write the *hexadecimal* representation of the following MOVZ (Move Conditional on Zero) instruction after it is assembled. The instruction places the contents in GPR *rs* into GPR *rd* if the value in GPR *rt* is equal to zero.

- This instruction is a R-format instruction.
- The *opcode* and *shamt* field values are 0, and the function code is 10.
- In the instruction below, the number after R represents the register number, and R4 maps to *rs*.
- You must give the values of the fields, so we can clearly see what binary values are given for individual fields.
- The final hexadecimal representation must include all digits – i.e., do NOT ignore leading zeros. Also, put 0x as the prefix.

MOVZ R10, R4, R7

0x0087500A

Solution: Let us first convert the instruction to binary and then convert it to hexadecimal. R-format instructions take the following form:

op rs rt rd shamt funct

where op and funct are 6 bits, and all others are 5 bits. As a sanity check, this does sum to 32 bits. We are given that op=000000, shamt=00000, funct=001010. For the registers, rd=01010 (R10), rs=00100 (R4), rt=00111 (R7). Thus, we have the following binary code

000000 00100 00111 01010 00000 001010

Now let's split this into 4 bit chunks

0000 0000 1000 0111 0101 0000 0000 1010

Converting each of these chunks to hex yields our answer 0x0087500A.

## Q2. Basic MIPS 5-stage pipeline

Consider the following MIPS code:

```
I1: DADD    R8,   R10, R17
I2: LD      R10, 100(R8)
I3: SD      R8,   200(R8)
I4: DADDI   R8,   R10, 4
```

Suppose we have the simple MIPS 5-stage pipeline from Appendix C in the book (i.e., every instruction must go through all 5 stages).

- (a) Fill in the two pipeline timing charts that show the execution of the instructions. For the first chart, assume that no forwarding is available, except the in-cycle forwarding – i.e., the register file will be written in the first half of a clock cycle and be read in the second half of the clock cycle. For the second chart, assume that forwarding is available in the pipeline.

### • Without forwarding

|    | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|----|---|---|---|---|---|---|---|---|---|----|----|----|----|
| I1 | F | D | E | M | W |   |   |   |   |    |    |    |    |
| I2 |   | F | D |   |   | E | M | W |   |    |    |    |    |
| I3 |   |   | F | D |   |   | E | M | W |    |    |    |    |
| I4 |   |   |   | F | D |   |   |   | E | M  | W  |    |    |

### • With forwarding

|    | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|----|---|---|---|---|---|---|---|---|---|----|----|----|----|
| I1 | F | D | E | M | W |   |   |   |   |    |    |    |    |
| I2 |   | F | D | E | M | W |   |   |   |    |    |    |    |
| I3 |   |   | F | D | E | M | - |   |   |    |    |    |    |
| I4 |   |   |   | F | D | E | M | W |   |    |    |    |    |

(b) **(Will NOT be graded)** Represent timing charts in the previous question in pipelining timing tables.

• **Without forwarding**

| Ins |    | IF | ID | EX | MEM | WB | Comment |
|-----|----|----|----|----|-----|----|---------|
| (1) | I1 | 1  | 2  | 3  | 4   | 5  |         |
| (2) | I2 | 2  | 3  | 6  | 7   | 8  |         |
| (3) | I3 | 3  | 4  | 7  | 8   | -  |         |
| (4) | I4 | 4  | 5  | 9  | 10  | 11 |         |

• **With forwarding**

| Ins |    | IF | ID | EX | MEM | WB | Comment |
|-----|----|----|----|----|-----|----|---------|
| (1) | I1 | 1  | 2  | 3  | 4   | 5  |         |
| (2) | I2 | 2  | 3  | 4  | 5   | 6  |         |
| (3) | I3 | 3  | 4  | 5  | 6   | -  |         |
| (4) | I4 | 4  | 5  | 6  | 7   | 8  |         |

- (c) Show and classify all the data dependencies: (*true, anti, output*).

The MIPS code is duplicated below for your reference.

```
I1: DADD    R8,   R10, R17
I2: LD      R10,  100(R8)
I3: SD      R8,   200(R8)
I4: DADDI   R8,   R10, 4
```

Note that we discussed in class that WAR (from anti-dependences) and WAW (from output dependences) hazards cannot happen in the basic 5-stage MIPS pipeline. In this question, you will just identify the dependences.

You are also required to indicate which of the potential RAW data hazards are eliminated because of forwarding. (You may write `Eliminated` next to the true dependence applied.)

Represent each dependence in the following format:

$$I_x \rightarrow I_y \text{ on register}$$

For example,  $I_7 \rightarrow I_8$  on R10

**Warning! Dependences incorrectly represented won't be graded.**

True dependences:

I1 --> I2 on R8 (Eliminated)  
I1 --> I3 on R8 (Eliminated)  
I2 --> I4 on R10 (Eliminated)

Anti dependences:

I1 --> I2 on R10  
I2 --> I4 on R8  
I3 --> I4 on R8

Output dependences:

I1 --> I4 on R8